Desk Number _____

Student Number _____

Student Name _____

School of Science and Engineering

MIDTERM EXAMINATION

Term 1 2025-2026

CSC1001 Introduction to Computer Science

Examination Duration: 120 minutes

This examination has __5__ questions.

The score distribution of the 5 question is Question 1(40%), Question 2(20%), Question 3(10%), Question 4(15%), Question 5(15%)

Exam Conditions:

This is a FORMAL Examination

This is a RESTRICTED OPEN BOOK Exam. Maximum of one (1) sheet of A4-sized double-sided notes is permitted

Materials Permitted:

Maximum of one (1) sheet of handwritten notes double sided are permitted. **NO OTHER MATERIALS PERMITTED**

Calculators without the functionalities of programming and file storage are permitted.

Materials Supplied:

1 × 6 Page Answer Booklet

You must write all your answers in your answer book.

Question 1. (40%)

Please describe what will be printed for each program as below.

	Program	Printed Results
1) 3%	`x = (2**3 % 5) // 2 + float("77".replace("7", "1"))` `print(x)`	12.0 0% without decimal point
2) 3%	`s1, s2, s3 = eval("15-3"), str(15-3), "15-3"` `print(s1 + len(s2) + len(s3))`	18
3) 3%	`t = [10, 20, "python", False, 80]` `print(t[2][2:5].upper(), t[0:2])`	THO [10, 20] 1% without []

4) 3%	`x = 10`
	`y = []`
	
	`if (not x > 5) or (y and x % 2 == 0):`
	`  print("csc1001")`
	`elif len(y) == 0 and x > 8:`
	`  print("hello")`
	`else:`
	`  print("world")`
		hello
		1% for "hello"

5) 3%	`n = 12`
	`i = 1`
	`while i < n:`
	`  print(i, end="*")`
	`  i = i * 2`
		1*2*4*8*

6) 4%	`cache = []`
	`def log_event(event, max_size=2):`
	`  global cache`
	`  cache.append(event)`
	`  if len(cache) > max_size:`
	`  cache = cache[max_size * -1:]`
	
	`log_event("start")`
	`log_event("finish", max_size=2)`
	`log_event("end", max_size=2)`
	
	`print(cache)`
		['finish', 'end']
		1% without []
		Both single and double quotation marks are acceptable

7) 4%	`data = "student_ID_12345.csv"`
	`course = ""`
	`recording = False`
	`for char in data:`
	`  if char == '_':`
	`  recording = not recording`
	`  continue`
	`  if char == '.':`
	`  break`
	`  if recording:`
	`  course += char`
	`  print(course)`
		I
		ID

8) 3%	`for i in range(3):`
	`  if i == 2:`
	`  print(i, end=",")`
	`  break`
	`  else:`
	`  print("Loop finished")`
		Loop finished
		Loop finished
		2,
		1% without comma

9) 3%	`s = "user-tongxin-li-admin@example.com"` `print(s.split("@")[0].split("-")[-1])`	admin

10) 4%	`counts = {'a': 5, 'b': 10}`
	`counts['c'] = counts.get('c', 0) + 1`
	`print(counts['a'], counts['c'])`
	`print('b' in counts)`
		5 1
		True
		2% for true with a lower case "t"

11) 3%	`h = open('example.txt', 'w')`
	`h.write('Hello')`
	`h.close()`
	`h = open('example.txt', 'r')`
	`s = h.read()`
	`print('First read:', s)`
	`s = h.read()`
	`print('Second read:', s)`
		First read: Hello
		Second read:
		1% if missing either line of the answer

12) 4%	`d = {'b':1, 'a':2, 'c':0}`
	`lst = []`
	`for k, v in d.items():`
	`  if v > 0:`
	`  lst.append(k)`
	`  lst.sort()`
	`  print(lst)`
		['b']
		['a', 'b']
		1% without []
		Both single and double quotation marks are acceptable

Question 2. (5*4% = 20%)

Please answer the following short questions.

1) Provide the Binary and Hexadecimal representations of the decimal number 29.25.

Binary: 11101.01（2%）

Hexadecimal: 1D.4（2%）

2) Please list two advantages of using binary numbers in modern computer systems.

- *Easy to implement physically*
- *Simple calculation rules*
- *Easy to combine arithmetic and logic operations*
- *Against noise (for analog signal)*

Points can be awarded at marker's discretion if the meaning matches any two items，（2% for each）

3) Please describe the differences between arguments and parameters.

A parameter is a variable which we use in the function definition that is a 'handle' that allows the code in the function to access the arguments for a particular function invocation.

An argument is a value we pass into the function as its input when we call the function.

Parameter - function definition（1%）； variable（1%）

Argument – function call（1%）； value（1%）

Points can be awarded at marker's discretion if the meaning matches.

4) Will the following be implemented without causing an error? (Yes/No)

```
data = ("apple", 5, "orange")
print(data[0])
data[1] = 10
print(data)
```

Explain the difference between a list and a tuple in Python. Provide one reason to use a tuple instead of a list.

No.（1%）

Lists are mutable, while tuples are not.（1%）

Tuples are more efficient in terms of memory use and performance than lists. （2%）

Points can be awarded at marker's discretion if the meaning matches.

5) What is the purpose of the return keyword in a function? What happens if a function does not have a return statement?

The return statement ends the function execution and 'sends back' the result of the function.

Python will implicitly return None at the end of the function, if the return statement is omitted.

Ends the function （1%）；sends back result （1%）

Return None / Void Function （2%）；

Points can be awarded at marker's discretion if the meaning matches.

Question 3. (10%)

Write a Python program that plays a number-guessing game with a user. The game should follow these rules:

- The computer fixes a **secret integer** between 1 and 50. You can simply set this number as 10 in your program, for example.
- The user gets a maximum of **5** attempts to guess the number.
- After each guess, the program must provide feedback to the user, telling them if their guess was "**Too high**!" or "**Too low!**".
- If the user guesses correctly, the game ends, and the program prints a success message, like "**Correct! You guessed it in [N] attempts.**" where N is the total number of attempts.
- If the user runs out of attempts, the game ends, and the program should reveal the secret number, printing "**Sorry, you've run out of attempts. The secret number was [X].**" where X is the correct integer.

For syntax errors, points can be withheld at marker's discretion.

• Correctly fix an integer （1%）any one from 1 to 50

• Correctly set maximum of 5 attempts（1%）

• Correct input（2%）Note for the conversion from str to int

• Correct feedback output: "Too high!" or "Too low!" （2%）

• Correct output for a right guess: "Correct! You guessed it in [N] attempts."（2%）

• Correct output for a wrong guess: "Sorry, you've run out of attempts. The secret number was [X]."（2%）

Question 4. (15%)

Consider the Python's built-in string method **find()**. Let *mystr* be a non-empty string. In your answer, state **what value *mystr.find(tar)* returns** in **each** of the following cases:

(a) the target string *tar* occurs once in *mystr*;
(b) the target string *tar* occurs more than once in *mystr*;
(c) the target string *tar* does not exist in *mystr*.

(a) Index of the first character of that single occurrence.（2%）
(b) Index of the first character of the first occurrence (left-most are also acceptable).（2%）
(c) -1.（2%）

Then, write a function **myFind**(*main_string*, *sub_string*):

- *main_string* is the string to be searched.
- *sub_string* is the string to search for.
- The function should return the **ending index** of the first occurrence of *sub_string* within *main_string*.
- If *sub_string* is not found, the function should return -1.
- If *sub_string* is empty, the function should return 0.

You cannot use the built-in find() or index() methods in your implementation.

(e.g., if *main_string* is "hello world csc1001" and *sub_string* is "world", the result should be 10

if *main_string* is "" and *sub_string* is "", the result should be 0)

For syntax errors, points can be withheld at marker's discretion.

• Correctly return the ending index （2%）

• Solely return the ending index of first occurrence（2%）

• If *sub_string* is not found, the function should return -1.（2%）

• If *sub_string* is empty, the function should return 0.（1%）

• Correctly address loops (if any) and indices other than the final result（2%）

Question 5. (15%)

You are in charge of counting the votes for a class election. You have a file, **votes.txt**, where every line contains the name of the candidate who received a vote. All names are unique.

Example **votes.txt** content:
Alice
Bob

Alice
Charlie
Alice
Bob
Daniel
...

Please write a Python program that:

- Reads the **votes.txt** file.
- Counts the total number of votes for each candidate.
- Creates a new file named **results.txt** that lists each candidate and their total vote count, sorted by the vote count in a decreasing order.

(e.g., the **results.txt** should look like:

Alice 11
Bob 8
Charlie 7
Daniel 5
...

)

For syntax errors, points can be withheld at marker's discretion.

• Correctly open votes.txt（2%）

• Correctly read and store names（2%）

• Correct amount counting（4%）

• Correct sorting（4 分）

• Correctly open results.txt（1%）

• Correctly write names and the corresponding amounts（2%）Note: must have line break "\n"

END OF EXAMINATION